Expense Tracker
Project

Step-by-Step Coding Guide

HTML | CSS | JavaScript
Beginner to Intermediate Level

What You'll Build

A fully functional Expense Tracker web application with:

- Budget management and tracking
- Add and remove expenses dynamically
- Category-wise spending reports
- Real-time budget status updates

Part — Table of Contents

#	Section	Steps
Part 1	Create the HTML Structure	Steps 1–5
Part 2	Create JavaScript Variables	Steps 1–2
Part 3	Create setBudget Function	Steps 1–3
Part 4	Create addExpense Function	Steps 1–6
Part 5	Create removeExpense Function	Steps 1–3
Part 6	Create renderTable Function	Steps 1–10
Part 7	Create updateBudgetStatus Function	Steps 1–6
Part 8	Create renderCategoryReport Function	Steps 1–7

Part 1 — Create the HTML Structure

In this part, we will create the basic HTML elements: heading, budget section, expense form, expense table, and report section.

Step 1 — Create Main Heading

HTML

```
<h2>Expense Tracker</h2>
```

What This Does

- Displays the title of the project on the webpage.

Step 2 — Create Budget Section

HTML

```
<label>Monthly Budget: </label>
<input type="number" id="budgetInput" value="5000" />
<button onclick="setBudget()">Set Budget</button>
<p id="budgetStatus"></p>
```

What This Does

- Creates an input field for entering the budget.
- Creates a button to save the budget.
- Displays remaining budget or warning messages.

Step 3 — Create Expense Input Form

HTML

```
<input type="text" id="expenseName" placeholder="Expense name" />
<input type="number" id="expenseAmount" placeholder="Amount" />
<input type="text" id="expenseCategory" placeholder="Category" />
<button onclick="addExpense()">Add</button>
```

What This Does

- Takes expense details from the user.
- Button calls the addExpense() function.

Step 4 — Create Expense Table

HTML

```
<table id="expenseTable">
  <tr>
    <th>Name</th>
    <th>Amount</th>
    <th>Category</th>
    <th>Action</th>
  </tr>
</table>
```

What This Does

- Creates a table to display expenses.
- "Action" column will contain the Remove button.

Step 5 — Create Total and Report Section

HTML

```
<p id="totalSpent"></p>
<div id="categoryReport"></div>
```

What This Does

- Displays total amount spent.
- Displays category-wise spending report.

Part 2 — Create JavaScript Variables

Step 1 — Create Budget Variable

JAVASCRIPT

```
let budget = 5000;
```

What This Does

- Stores the monthly budget value.

Step 2 — Create Expenses Array

JAVASCRIPT

```
let expenses = [];
```

What This Does

- Stores all expense objects inside an array.

Part 3 — Create setBudget Function

Step 1 — Create Function

JAVASCRIPT

```
function setBudget() {

}
```

What This Does

- Creates a function to update the budget.

Step 2 — Read Budget Input

JAVASCRIPT

```
budget = Number(document.getElementById("budgetInput").value);
```

What This Does

- Gets value from the input field.
- Converts string into number.
- Stores value inside budget.

Step 3 — Update Budget Status

JAVASCRIPT

```
updateBudgetStatus();
```

What This Does

- Calls another function to refresh remaining budget display.

Part 4 — Create addExpense Function

Step 1 — Create Function

JAVASCRIPT

```
function addExpense() {

}
```

What This Does

- Creates a function to add expenses.

Step 2 — Get User Inputs

JAVASCRIPT

```
let name = document.getElementById("expenseName").value.trim();
let amount = Number(document.getElementById("expenseAmount").value);
let category = document.getElementById("expenseCategory")
               .value.trim() || "Other";
```

What This Does

- Reads all input values.
- Removes extra spaces using trim().
- Adds default category "Other" if empty.

Step 3 — Validate Inputs

JAVASCRIPT

```
if (name && amount > 0) {

}
```

What This Does

- Checks: Name should not be empty.
- Checks: Amount should be greater than 0.

Step 4 — Add Expense Object

JAVASCRIPT

```
expenses.push({ name, amount, category });
```

What This Does

- Adds a new expense object into the expenses array.

Example object:

JAVASCRIPT

```
{ name: "Food", amount: 500, category: "Grocery" }
```

Step 5 — Clear Input Fields

JAVASCRIPT

```
document.getElementById("expenseName").value = "";
document.getElementById("expenseAmount").value = "";
document.getElementById("expenseCategory").value = "";
```

What This Does

- Clears old input values after adding expense.

Step 6 — Refresh Table

JAVASCRIPT

```
renderTable();
```

What This Does

- Updates table with new expense data.

Part 5 — Create removeExpense Function

Step 1 — Create Function

JAVASCRIPT

```
function removeExpense(index) {

}
```

What This Does

- Creates a function to delete expenses.

Step 2 — Remove Expense

JAVASCRIPT

```
expenses.splice(index, 1);
```

What This Does

- Removes 1 item from array using its index position.

Step 3 — Refresh Table

JAVASCRIPT

```
renderTable();
```

What This Does

- Updates table after deleting expense.

Part 6 — Create renderTable Function

Step 1 — Create Function

JAVASCRIPT

```
function renderTable() {

}
```

What This Does

- Creates a function to display all expenses in the table.

Step 2 — Get Table Element

JAVASCRIPT

```
let table = document.getElementById("expenseTable");
```

What This Does

- Accesses the HTML table.

Step 3 — Reset Table Content

JAVASCRIPT

```
table.innerHTML = `
<tr>
  <th>Name</th>
  <th>Amount</th>
  <th>Category</th>
  <th>Action</th>
</tr>
`;
```

What This Does

- Clears old table rows.
- Adds table headings again.

Step 4 — Create Total Variable

JAVASCRIPT

```
let total = 0;
```

What This Does

- Stores total amount spent.

Step 5 — Loop Through Expenses

JAVASCRIPT

```
expenses.forEach((exp, i) => {

});
```

What This Does

- Loops through every expense object.

Step 6 — Add Expense Amount

JAVASCRIPT

```
total += exp.amount;
```

What This Does

- Adds each expense amount into total.

Step 7 — Create Table Rows

JAVASCRIPT

```
table.innerHTML += `
<tr>
  <td>${exp.name}</td>
  <td>${exp.amount}</td>
  <td>${exp.category}</td>
  <td>
    <button onclick="removeExpense(${i})">
      Remove
    </button>
  </td>
</tr>
`;
```

What This Does

- Dynamically creates rows inside the table.
- Adds Remove button for each expense.

Step 8 — Show Total Spent

JAVASCRIPT

```
document.getElementById("totalSpent").textContent =
  `Total Spent: ${total}`;
```

What This Does

- Displays total amount spent.

Step 9 — Update Budget Status

JAVASCRIPT

```
updateBudgetStatus(total);
```

What This Does

- Updates remaining budget.

Step 10 — Show Category Report

JAVASCRIPT

```
renderCategoryReport();
```

What This Does

- Displays category-wise expense report.

Part 7 — Create updateBudgetStatus Function

Step 1 — Create Function

JAVASCRIPT

```
function updateBudgetStatus(totalSpent = 0) {

}
```

What This Does

- Creates a function with default parameter value of 0.

Step 2 — Calculate Remaining Budget

JAVASCRIPT

```
let remaining = budget - totalSpent;
```

What This Does

- Calculates how much money is left.

Step 3 — Access Status Paragraph

JAVASCRIPT

```
let status = document.getElementById("budgetStatus");
```

What This Does

- Accesses the paragraph used to display budget messages.

Step 4 — Check Over Budget

JAVASCRIPT

```
if (remaining < 0) {

}
```

What This Does

- Checks if spending crossed the budget limit.

Step 5 — Show Warning Message

JAVASCRIPT

```
status.textContent =
  `Over Budget! Remaining: ${remaining}`;
status.className = "over-budget";
```

What This Does

- Displays warning message.
- Applies red color styling.

Step 6 — Show Normal Budget Message

JAVASCRIPT

```
status.textContent =
  `Remaining Budget: ${remaining}`;
status.className = "";
```

What This Does

- Displays remaining budget normally.

Part 8 — Create renderCategoryReport Function

Step 1 — Create Function

JAVASCRIPT

```
function renderCategoryReport() {

}
```

What This Does

- Creates a function to show category-wise spending.

Step 2 — Create Empty Object

JAVASCRIPT

```
let categoryTotals = {};
```

What This Does

- Stores category-wise totals.

Step 3 — Loop Through Expenses

JAVASCRIPT

```
expenses.forEach((exp) => {

});
```

What This Does

- Visits every expense object.

Step 4 — Add Category Totals

JAVASCRIPT

```
categoryTotals[exp.category] =
  (categoryTotals[exp.category] || 0) + exp.amount;
```

What This Does

- Adds amounts category-wise.
- Creates category if it does not exist.

Step 5 — Access Report Div

JAVASCRIPT

```
let reportDiv =
  document.getElementById("categoryReport");
```

What This Does

- Accesses the report section.

Step 6 — Add Report Heading

JAVASCRIPT

```
reportDiv.innerHTML = "<h4>Category Report</h4>";
```

What This Does

- Displays report title.

Step 7 — Display Categories

JAVASCRIPT

```
for (let cat in categoryTotals) {
  reportDiv.innerHTML +=
    `<p>${cat}: ${categoryTotals[cat]}</p>`;
}
```

What This Does

- Displays each category and its total spending.

Congratulations!

You have completed the Expense Tracker project! You now know how to build a complete web application using HTML and JavaScript. Try adding your own features like date filters, editing expenses, or saving data to localStorage.